POCKET RELAY MINER · V0.1.3 · STANDALONE MODE · 1 PROCESS · 2026-10-13

Relay Miner standalone capacity

One **standalone** process, the relayer and the miner in 1 binary with its state in an embedded store on local disk. It served **10,548,118** relays, ~2,400 relays/s over the 73 minutes of load (2,420 relays/s in steady state), and the chain settled **1,500 of 1,500** claims, each after its proof, none expired or slashed, even though the process was killed on purpose 2 times, once in a proof window and once in a claim window.

How it was tested

The same load and tests as the v0.1.0 report, with the differences listed below. Everything after this section refers to this setup; it is not repeated.

4 terms used throughout. A *session* is 1 (application, service) pair served by 50 suppliers, and each supplier keeps 1 Merkle tree (SMST) of it in the store and submits 1 claim for it. The 6 sessions born in the same block are a *generation*: 6 × 50 = 300 trees and 300 claims. A tree is *live* from its first relay until the miner compacts it at the claim. After a session ends, its claim is submitted during the claim window, and its proof during the proof window that follows. Heights are written h31 for height 31.

SETUP · CONTAINER LIMITS

Component	Limits	What it does
standalone	16 GiB · 20 CPUs GOMEMLIMIT 14 GiB	the relayer side validates ring signatures, charges the stake, signs responses and hands each relay to the miner side; the miner side builds the SMST trees and submits claims and proofs
store	local volume	all session state, in an embedded key-value store with a write-ahead log (WAL) synced to disk about once a second
validator	60 s configured ~59.6 s measured	the chain (pocketd 0.1.35), reset to genesis before the run
load client	6 pods 1,200 millicores · GOMAXPROCS=2 each	the project's own CLI, sending relays straight to the process and verifying every response

Load. ~2,400 relays/s over 3 transports (jsonrpc, websocket, grpc), from h6 to h81. This is the load chosen for the test, not a measured ceiling: the process was never CPU-throttled, and its CPU, averaged per block, peaked at 5.72 of its 20 cores. 6 services, 2 per transport, each

driven by 1 application across the 50 suppliers of its session. The client verifies every response; its per-pod results were not kept, so "served" here is the process's own count.

1 MiB burst. At h44 a second client sent 1 MiB bodies to develop-http, 128 at a time, for 15 seconds, on top of the full load.

Kills. kill -9 sent from the Kubernetes node, the signal a kernel OOM kill sends: once in a proof window (h63), with 6 sessions of the sampled supplier proving, and once in a claim window (h91), with none of its generation's 300 claims sent yet. The 2 kills hit 2 different generations.

Where the method differs.

1. **The client is paced.** Each pod runs at concurrency 16 (instead of 8 in a closed loop) and asks for 420 relays/s, and it skips a tick when all 16 slots are busy, so the load held ~400 relays/s per pod. The client pods have 1,200 millicores and GOMAXPROCS=2, so that the client is not the limit.
2. **The claim-window kill ran without load, on the generation ending at 80.** An attempt under load, in the claim window of the generation ending at 60, found its 300 claims all sent between 2 polls 0.5 s apart (they landed in block 73) and killed nothing. The kill was re-armed for the generation ending at 80 and fired at h91, the height at which the tools treat its claim window as open, after the load had stopped at h81. Heights in this report are the chain heights the tools saw when they acted; the chain recorded that generation's claims at h95.
3. **Fewer applications staked.** 6 applications were staked, 1 per service, each with 10M POKT, so stake never ran out; the load started at h6.
4. **Counters are read from the process itself.** One process holds both sides, so a kill resets the relayer side's counters too. Every counter was read from the process's own metrics endpoint once per height, and again just before each kill (in the same node shell that sent it) and after the process was ready. Container memory and CPU throttling, and the 1-minute rate of the process's served counter, come from the cluster's metrics scraper.
5. **The relay identity is checked per process lifetime**, from those reads, instead of once for the whole run.
6. **Disk was not a constraint.** The test cluster does not enforce the volume's 10 GiB request, and free disk never fell below 921.5 GB, so the 1 GiB floor below which new work stops was not reached.

One setup. These figures come from one topology on one machine. Supplier count, services, validation mode, transports and hardware all change them. Use them as a reference for sizing your deployment, not as a guarantee.

Steady state

Heights 20–42: full load, before the burst. Memory is the highest reading; CPU, throughput and store rates are the mean.

STEADY STATE · HEIGHTS 20–42

What	Value	Read it as
relays per second	2,420	2,161 to 2,480 by height
CPU	4.35 cores	highest 5.34 cores, at the claim of h33; limit 20
memory, container working set, peak	3,878 MiB	mean 2,822 MiB; limit 16 GiB
memory, process RSS, peak	3,527 MiB	the process's resident memory, from its own metrics
Go heap in use, peak the process's own heap gauge	2,960 MiB	GOMEMLIMIT 14 GiB
store on disk, peak	2,448 MiB	the trees of the live sessions, before compaction
tree writes to the store	28,352 /s	11.7 per relay served (node sets and deletes)
WAL syncs	1.0 /s median mean 1.7, with the claims' bursts	the store's fsync of its write-ahead log to disk, not request latency: p99 17.0 ms, mean of the per-height p99. Requests: see Latency

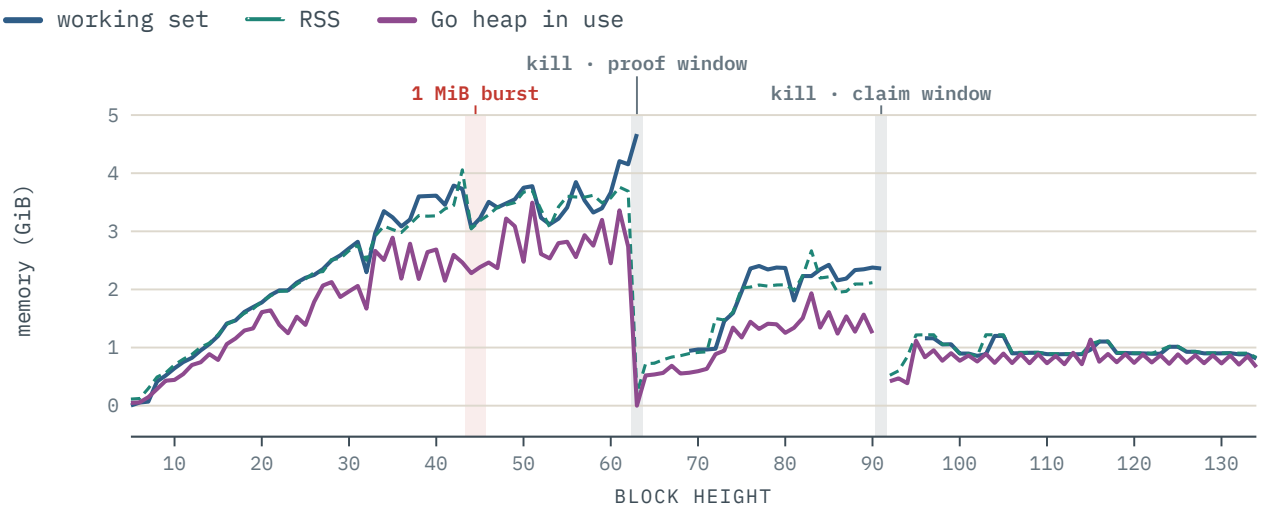
The tree writes per relay grow with the trees: they start near 4 when a generation is born, and reach ~14 when its 20 blocks of relays are in. After the chain raised the relay-mining difficulty (see Sizing), they fell to 3.7 per relay over h65–81.

Height by height

The red band is the 1 MiB burst; grey marks the 2 kills.

CHART 1

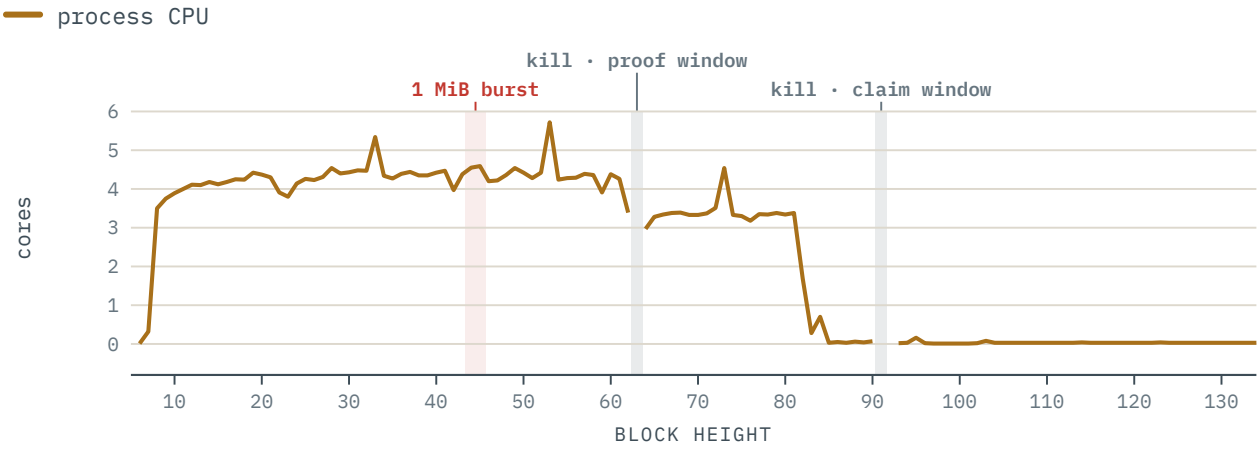
Memory



The limit (16 GiB) and GOMEMLIMIT (14 GiB) are far above this scale. Memory follows the live trees: it rises while 2 generations are served and falls at each claim. At the proof-window kill the last working-set reading of the killed process is 4,786 MiB, the highest of the run, and its own metrics read in the same shell as the kill show 4,365 MiB RSS and 2,609 MiB of heap. After each kill the container metric kept reporting the killed process for some blocks (h64–68 and h92–95); those heights are left as gaps. RSS comes from the running process and has no such lag.

CHART 2

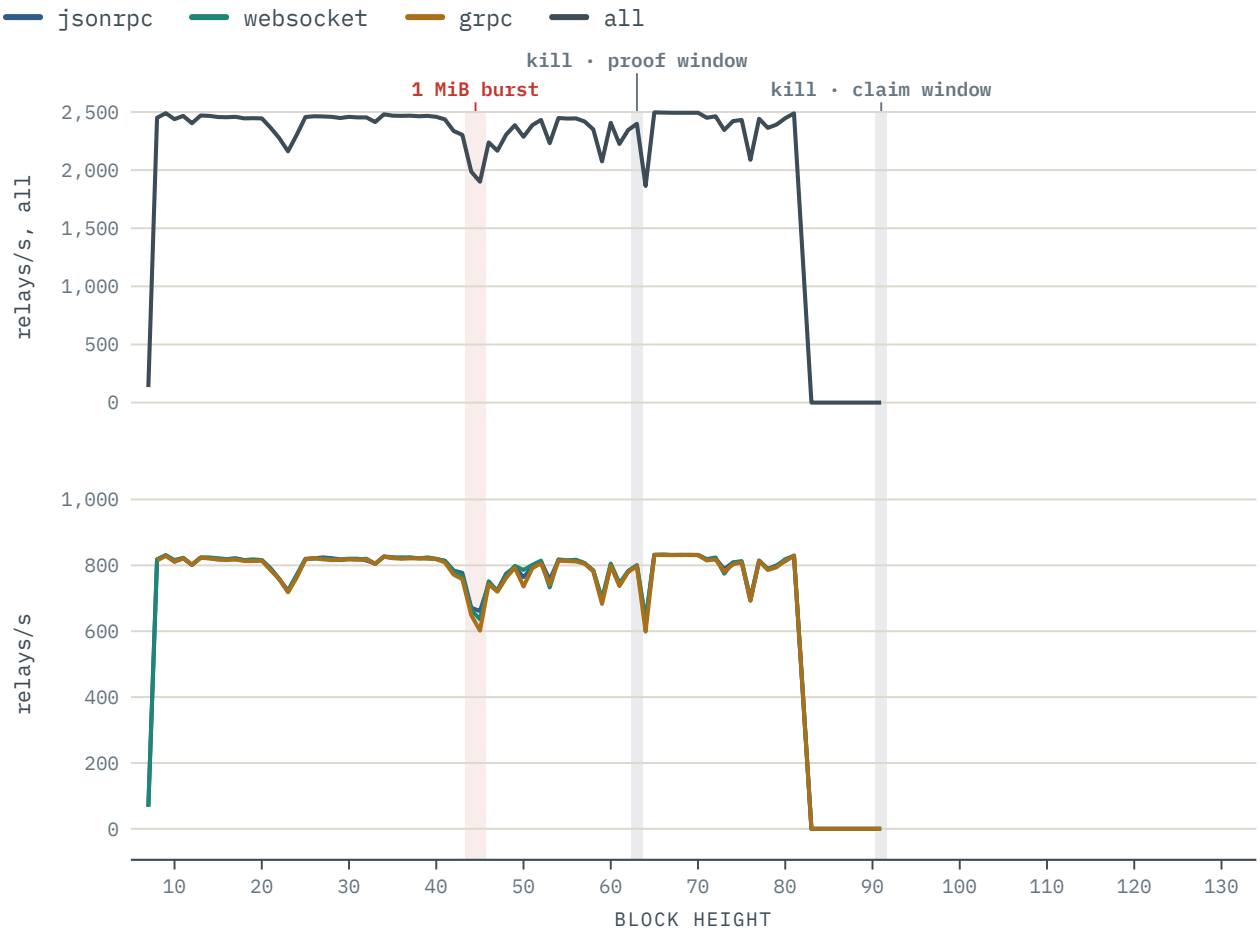
CPU



CPU follows the relay rate. Each claim adds a spike (h33, h53, h73). From h62 the chain's higher difficulty lets ~70 % of the relays skip the tree, and CPU is lower: 3.42 cores at 2,426 relays/s over h65–80, against 4.16 cores at 2,338 relays/s over h55–62. With no load (from h82) it is near 0.

CHART 3

The load, in total and by transport



790, 789 and 784 relays/s on average over h20–60 for jsonrpc, websocket and grpc: each transport was served at the rate its paced pods asked for. These are rates served, set by the client, not ceilings. The dip in the h44 reading comes before the burst, while the proofs of the generation ending at 20 were being sent (see Latency); the h45 reading is the burst, and h64 the proof-window kill. There is no load at the claim-window kill (h91).

Latency

Request latency is the time from the request reaching the process to the response being sent, from the process's own histogram: a median of 1.4 ms and a p99 of 7.3 ms per block in steady state, and a p99 of 607.7 ms in the block of the 1 MiB burst.

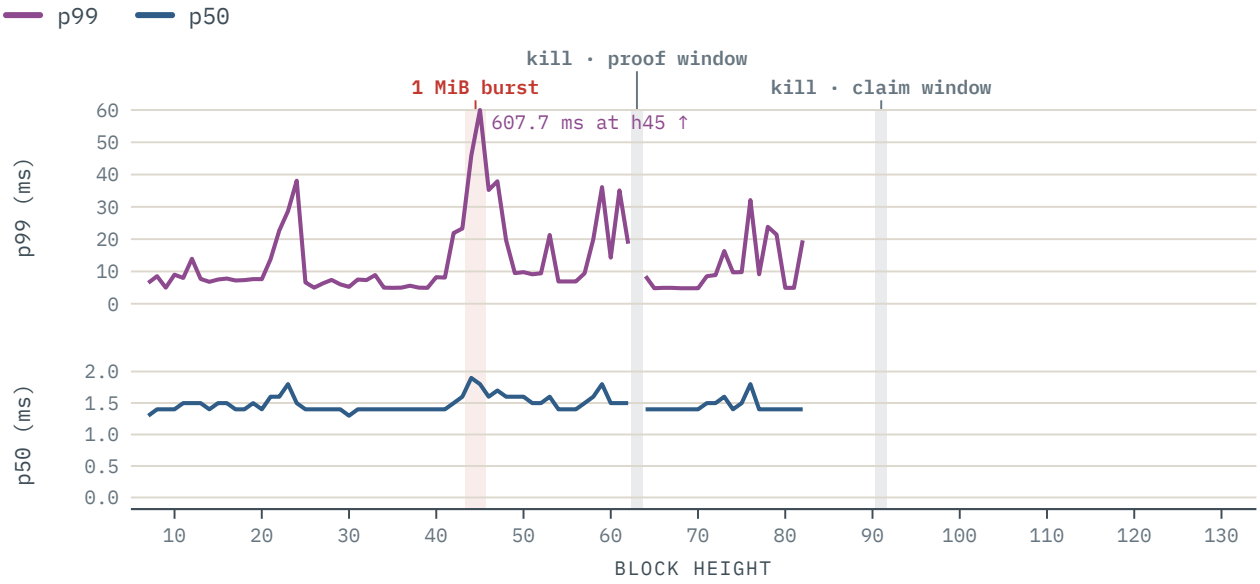
Each block's distribution is the difference between the histogram read at height N−1 and at N, both from the same process. Its quantiles are **bucket-interpolated**: linear inside the histogram bucket that holds them, whose bounds are 1, 2, 5, 10, 25, 50, 100, 250, 500 and 1,000 ms and up. A p99 of 607.7 ms means "inside the 500–1,000 ms bucket", not a measurement to the millisecond. The histogram covers jsonrpc and grpc (4 of the 6 services, ~95,466 relays per block); it has no websocket series. It counts served relays only, so the refused ones are not in it.

REQUEST LATENCY PER BLOCK · JSONRPC AND GRPC

Blocks	p50	p99	What happened
h20–42, steady	1.4 ms 1.3 to 1.8	7.3 ms 4.9 to 38.1	median of the 23 blocks, and their range
h21–24		13.8 · 22.7 · 28.7 · 38.1 ms	after a generation's birth at h21. Not explained: WAL syncs, flushes, compactions and CPU are ordinary; GC pauses sum to 36, 22 and 23 ms in h22–24 (median 11.1 ms), a few ms per cycle, short of these p99s
h33 · h53 · h73, claims		8.9 · 21.3 · 16.3 ms	the blocks where a generation's 300 claims landed
h43 · h44, proofs		23.3 · 45.8 ms	the proofs of the generation ending at 20 were built and sent: 218 and 802 WAL syncs against ~59 steady, 156.2 ms of GC pauses in h44. The h44 block ended 1 s before the burst
h45, the burst	1.8 ms	607.7 ms	the 15 s burst falls in this block; 288.9 ms of GC pauses
h46 · h47, after		35.2 · 37.9 ms	back under 50 ms one block after the burst
h65–81, higher difficulty		8.5 ms	median per-block p99
kills	h63 (2 processes) and h92 (no read at h91, during the restart) are left out. h64, the first block after the proof-window kill: p99 8.5 ms		

CHART 4

Request latency per block



Bucket-interpolated, jsonrpc and grpc; p99 capped at 60 ms, the burst's block named above it. Gaps: the kills; no requests after h82.

WHERE THE BURST'S P99 COMES FROM · P99 BY SERVICE, H43 → H45

Service	Transport	h43	h45
develop-http the 1 MiB service	jsonrpc	16.9 ms	976.0 ms
develop-http-eager	jsonrpc	22.5 ms	233.9 ms
develop-grpc	grpc	24.7 ms	268.3 ms
develop-grpc-optimistic	grpc	24.9 ms	302.7 ms

The process splits an HTTP relay's time into before the backend call (reading the body, metering, building the request, and validating in eager mode), the backend call, and after it (signing, compressing, writing the response, handing the relay to the miner side). In the burst's block their p99s on develop-http are 277.2, 821.3 and 202.5 ms, and on develop-http-eager 12.4, 226.2 and 2.0 ms. p99s do not add up, so the shares come from the means: 54.7 ms per develop-http relay (1.5 in h43), of which 8.9 before, 40.1 in the backend and 5.6 after. The backend call is the largest share and also slowed on develop-http-eager (mean 8.6 ms). The process's own time grew from 0.1 to 14.6 ms on the 1 MiB service and from 1.4 to 2.4 ms on develop-http-eager. The validation queue refuses when full rather than delaying (31 here), and develop-http's optimistic validation runs after the response (p99 164.1 ms). grpc has no such split: its 287.0 ms is not broken down.

The 1 MiB burst

The process survived the burst without a restart. It refused 31 relays on the burst's service. grpc and websocket fell in the h45 reading and were back within 20 relays/s of h43 in h46.

1 MIB BURST · H44

What	Value
relays refused, validation queue full (develop-http)	31
other refusals between the h44 and h45 readings	94 client gone · 1 backend network error (develop-http) 2 unreadable bodies, service unknown
throughput, h45 (the burst) vs h43	1,900 (h45) vs 2,303
grpc and websocket, h43 → h45	758 → 602 · 769 → 636
memory, working set h44 → h45	3,136 → 3,299 MiB
restarts	0
store fsync to disk (WAL sync) p99, highest of the run; not request latency	710 ms at h47 requests in the same block: p99 37.9 ms
request latency p99, the burst's block	607.7 ms (h45) see Latency

Each service's validation queue holds at most 128 MiB of requests waiting to be validated. 128 bodies of 1 MiB in flight are enough to fill develop-http's, and the process refused 31 develop-http relays as full instead of growing past it; the label does not tell burst bodies from the service's regular load. The burst client is stopped after 15 s with requests still in flight, and the 94 "client gone" are consistent with that. All 128 refusals fall between the h44 and h45 readings, and none came from the store.

> **The burst's memory peak was not sampled.** Memory is read once per height: the h44 reading was taken 1 s before the burst began, and the h45 reading 42 s after it ended. The figures above bound the burst without showing its peak. What is measured is that the process kept running: its restart count stayed at 0 until the proof-window kill.

The 2 kills

Each kill was followed by a restart in the same pod, back to ready in 13 s, and each generation it hit was claimed, proved and settled in full.

KILL -9 FROM THE NODE

Window	Work in flight when killed	Restart	Ready after	Its generation on chain
proof · h63	6 sessions of the sampled supplier proving, under load	0 → 1	13 s	ending at 40: 300 · 300 · 300
claim · h91	0 of the generation's 300 claims sent, no load	1 → 2	13 s	ending at 80: 300 · 300 · 300

"300 · 300 · 300" is claims · proofs · settled, from the chain's events. After the claim-window kill the restarted process submitted all 300 claims, which the chain recorded at h95, in the same claim window.

Sizing standalone mode

Size the process by CPU for the relay rate, by memory for the sessions it holds, and by disk for the same sessions. The rates below are what 1 process served under the test load, not its ceiling.

WHAT TO EXPECT · 10,548,118 RELAYS SERVED · HIGHEST READING OF THE RUN UNLESS STATED

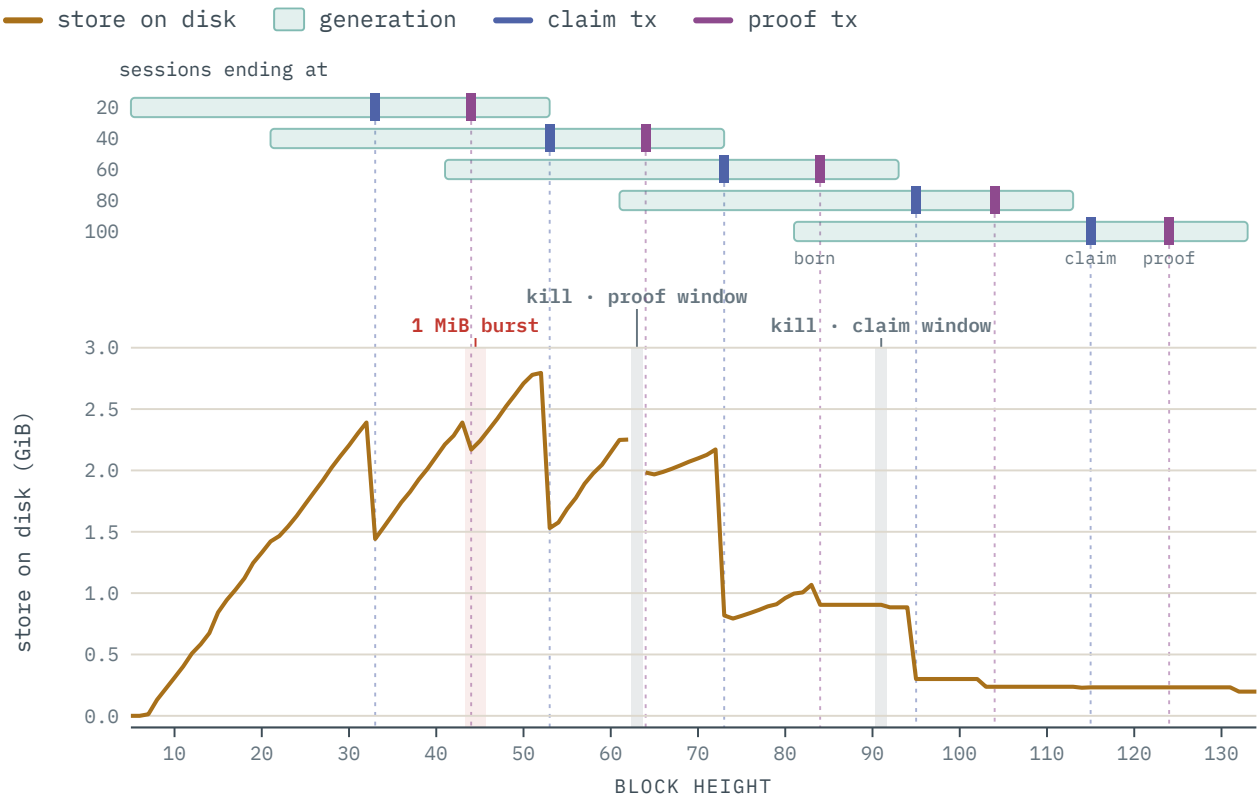
Resource	At ~2,400 relays/s	Read it as
CPU	4.35 cores steady 5.72 at a claim (h53)	the relays set it, every response signed; each generation's claims add a spike once every 20 blocks
memory	≤ 3,878 MiB steady 4,786 MiB: last reading before the proof-window kill	container working set, against the 16 GiB limit; the kill reading's instant is not known. It follows the live trees (600 at the peak: 12 sessions of the sampled supplier × 50 suppliers) and falls at every claim
memory, process	RSS ≤ 4,153 MiB 4,365 MiB at the proof-window kill	Go heap in use ≤ 3,576 MiB (h51), below GOMEMLIMIT 14 GiB

Resource	At ~2,400 relays/s	Read it as
disk	≤ 2,862 MiB (h52)	released at every claim; 202 MiB once every generation settled. New work stops below 1 GiB free, so leave room above the peak
disk writes	28,352 tree writes/s	WAL fsync to disk ~1 time a second, not request latency: p99 17.0 ms steady, 710 ms at h47 (requests: 37.9 ms); 0 failures

The memory a burst of large requests adds on top of this was not sampled (see the burst). No limit in this run was approached: the highest working set is 29 % of the 16 GiB limit and the per-block CPU peaked at 5.72 of 20 cores; sub-minute peaks were not measured.

CHART 5

The store on disk, with the generations on the axis



The 5 lanes at the top are the 5 generations, each running from birth to settlement with its claim and proof marked at the heights the chain recorded them. The curve is measured. The store grows ~97 MiB per height under load and drops at each claim, when the miner compacts the claimed trees; over h74–81 it grew ~30 MiB per height, because fewer relays enter a tree.

RELEASES AT EACH CLAIM

Heights	Store on disk, before → after	Generation claimed
32 → 33	2,448 → 1,476 MiB	ending at 20
52 → 53	2,862 → 1,566 MiB	ending at 40
72 → 73	2,223 → 838 MiB	ending at 60
94 → 95	905 → 307 MiB	ending at 80, after the claim-window kill, no load

Why the peak is at h52. For a session ending at height E, relays still arrive until E+10 and the claim window opens at E+12; these offsets are the v0.1.0 report's, from the same chain parameters, not re-queried here, and this run's claims landed on chain at E+13 to E+15, consistent with them. At h52 the generation ending at 40 is complete and its claim window has just opened, and the one ending at 60 has served 12 blocks: 1.6 sessions' worth of trees. That overlap repeats every 20 blocks and sets the peak, however efficient each tree is. A generation takes 33 blocks from the end of its session to settlement, about 33 minutes here.

After the first settlements the chain raised the relay-mining difficulty, for the sessions born from h61; relays skipped by difficulty jump in the h62 reading. Over the process's second lifetime (h63–h91), 70.6 % of the relays served no longer entered a tree (19.5 % over the whole run): the protocol stores only relays whose hash meets the difficulty target, and scales the reward to match. That is why CPU, tree writes and the store all grow more slowly from h62.

CHART 6

Store writes



Tree writes follow the relays that enter a tree. The WAL is synced about once a second, plus a burst of syncs at each claim and proof. The sync p99 is the store's fsync to disk, not request latency; under load it stays under 40 ms at 62 of 74 heights. 2 heights exceed the panel's 200 ms cap and are drawn at it: 710 ms at h47, after the burst, when requests had a p99 of 37.9 ms, and 244 ms at h114, at the last generation's claim, with no requests.

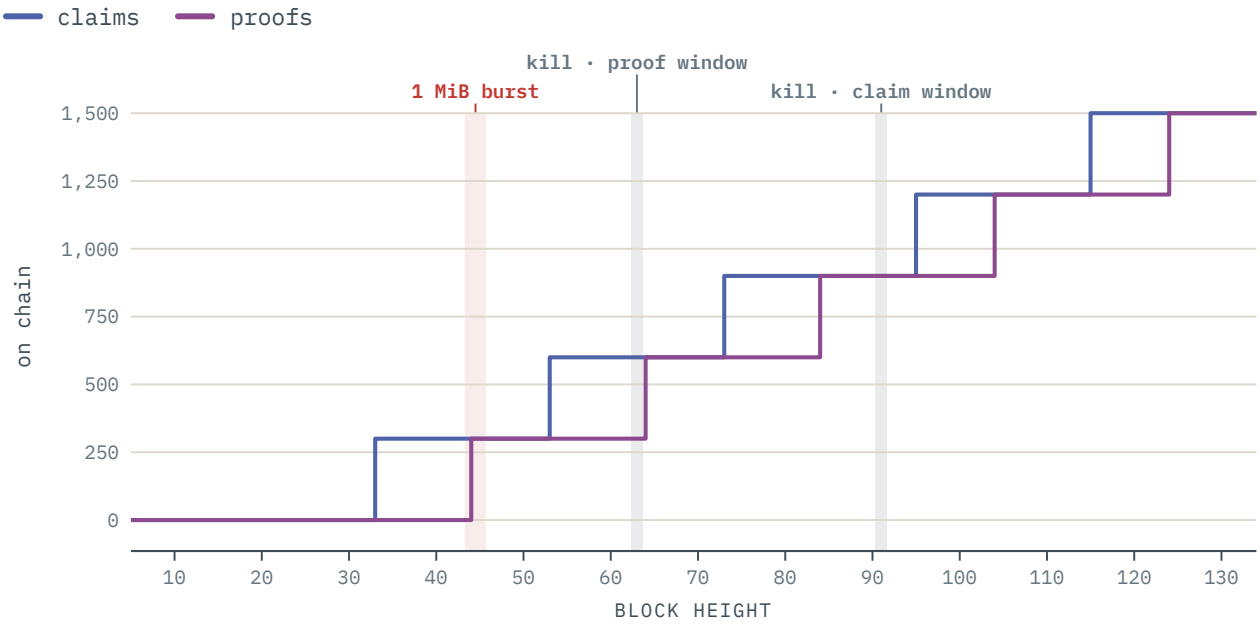
Every claim settled

Every claim the process made was proved and settled on chain, including the 2 generations hit by the kills. Counted by relay: the claims' relay totals add up to 8,486,694, against 8,486,647 relays the process counted as handed to its miner side. The chain has 47 more, which fits relays handed after the last counter read before a kill; it has none fewer. This compares totals, not claim by claim.

The figures below come from the chain's block results, not the process's counters, which reset with each restart.

CHART 7

Claims and proofs on chain



Cumulative EventClaimCreated and EventProofSubmitted, block by block. Each step is 1 generation, 6 sessions × 50 suppliers = 300 claims.

EACH GENERATION ON CHAIN · CLAIMS · PROOFS · SETTLED (HEIGHT)

Sessions ending at	Claims	Proofs	Settled
20	300 h33	300 h44	300 h53
40 proof-window kill	300 h53	300 h64	300 h73
60	300 h73	300 h84	300 h93
80 claim-window kill	300 h95	300 h104	300 h113

Sessions ending at	Claims	Proofs	Settled
100	300 h115	300 h124	300 h133

1,500 of 1,500 claims settled, none expired, none slashed, none discarded. The chain emitted 1,500 proof validity checks; their status field was not extracted, so validity is read from the settlements. The generation ending at 100 carries few relays: the load stopped at h81, 1 block after it was born.

> **And the relays.** Across 10,548,118 relays (1,750,289–1,762,131 per service), 5 were refused at h21 for a failed relay validation, well before either test, and the other 128 during the burst, as above. None was refused because of the store, and none after the proof-window kill.
>
> For each of the 6 services, relays served equal relays handed to the miner side plus relays skipped by difficulty, checked per process lifetime. The lifetime from h63 to the claim-window kill, the last with load, closes with a difference of 0 in every service; there was no load from h81 on, before that kill. The first lifetime, ended by the proof-window kill under load, closes with a difference of 2 (1 on develop-http, 1 on develop-websocket) out of 7.9 million: relays counted as served whose hand-off the killed process never recorded. The last lifetime, after the claim-window kill, served no relays.

What this run does not show

- **The peak memory of a burst of large requests.** The 15 s burst fell between 2 per-height readings.
- **A store near its disk limit.** Free disk never came near the 1 GiB floor, and the volume's size was not enforced.
- **A claim-window kill under load.** The one that fired ran with the load stopped.
- **Relays lost at a kill.** A relay served in the instant between the last read of the process's counters and the kill shows in no counter of this run. The chain records it only inside a claim's relay total, compared here in total, not claim by claim.
- **The store's refusal paths.** It was never short of disk and never failed a sync, so the paths that stop new work were not exercised.

Setup detail

The load. 1 indexed Kubernetes Job with 6 pods, 1 per service (develop-http, develop-http-eager, develop-websocket, develop-websocket-optimistic, develop-grpc, develop-grpc-optimistic), each re-running a 1,000,000-request load test in a loop. 6 applications, 1 per service, were staked with 10M POKT each, so stake never ran out. The response check covers HTTP status, the supplier's signature and the JSON-RPC error field.

```
pocket-relay-miner relay jsonrpc \
  --localnet --service develop-http \
  --load-test -n 1000000 --concurrency 16 --rps 420 --all-suppliers
```

The other pods differ only in the subcommand (`websocket`, `grpc`) and `--service`.

The process. Memory limit 16 GiB with GOMEMLIMIT 14 GiB, CPU limit 20, the store on a local persistent volume wiped before the run.

Source. 2026-10-13 23:16Z to 2026-10-14 01:29Z, v0.1.3 (572a72c), built from a clean tree, 1 image for the process and the load client.

Evidence. Every figure is read from the run's own artefacts: the per-height sampling, the process's own metrics read once per height and around each kill, the chain's block results for every height, the per-lifetime identity check, and the logs of the burst and of both kills.